

Damn Vulnerable DeFi, Shards Security Review

Prepared by: Khant Wai Yan Aung(SnavOhBurmaa)

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 21, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Shards Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Shards challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 21, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/shards/  
  ShardsNFTMarketplace.sol  
  IShardsNFTMarketplace.sol  
  ShardsFeeVault.sol
```

Protocol Summary

Shards is a marketplace that fractionalizes an NFT into many ERC1155 “shard” tokens and sells them for DVT. It has three pieces.

`ShardsNFTMarketplace.sol` is the marketplace. A seller lists an NFT with `openOffer`, splitting it into `totalShards` fractions at a fixed price (denominated in USDC). Buyers call `fill(offerId, want)` to buy `want` shards, paying DVT worked out from the offer price, the current DVT/USDC rate, and the total shards. A buyer can call `cancel(offerId,`

`purchaseIndex`) inside a timing window to reverse their purchase and get a refund. The DVT/USDC rate is maintained by an oracle account through `setRate`.

`IShardsNFTMarketplace.sol` holds the shared structs (`Offer`, `Purchase`) and the custom errors.

`ShardsFeeVault.sol` collects protocol fees.

In the test setup the offer price is 1_000_000e6 and the total shards is 10_000_000e18 , the marketplace holds about 750 DVT of seller fees, and a separate staking contract holds 100,000 DVT. The player starts with zero DVT. The goal is to drain the marketplace and send the funds to the recovery account, leaving staking untouched.

Roles

The seller opens an offer and lists an NFT split into shards. The buyer fills part of an offer and may cancel it within the timing window. The oracle is the only account allowed to update the DVT/USDC rate through `setRate`. Anyone can call `fill` and `cancel` for their own purchases, no special role is needed to buy.

Executive Summary

The marketplace pays and refunds a purchase with two different formulas, and the buy path rounds the price down to zero. Together they let a player who holds no DVT drain the marketplace fees in a single transaction.

`fill()` charges $\text{price} * \text{rate} / (1e6 * \text{totalShards})$ per shard and rounds down, so small buys cost nothing. `cancel()` refunds $\text{rate} / 1e6$ per shard, a value about $1e13$ times larger, because the refund formula forgot to scale by price and total shards. On top of that the cancel timing window is inverted, so a purchase can be cancelled in the same block. A player buys shards for free, cancels for a real refund paid out of the marketplace fees, then sizes a second buy so the refund equals the whole marketplace balance and pulls it out, all in one transaction.

Issues found

Severity	Number of issues found
High	1
Medium	2
Low	1
Informational	2
Total	6

ID	Title	Severity
[H-1]	Mismatch between the <code>fill()</code> price and the <code>cancel()</code> refund lets anyone drain the marketplace	High
[M-1]	<code>fill()</code> rounds the price down to zero, so shards can be bought for free	Medium
[M-2]	<code>cancel()</code> time window is inverted and does not match the documented behavior	Medium
[L-1]	Missing zero address check on the <code>oracle</code> in the constructor	Low
[I-1]	Unused custom errors in the interface	Informational
[I-2]	Magic numbers used instead of named constants	Informational

Findings

High

[H-1] Mismatch between the `fill()` price and the `cancel()` refund lets anyone drain the marketplace

Description:

When a buyer fills part of an offer, the DVT they pay is worked out like this:

```
// src/shards/ShardsNFTMarketplace.sol:135-137
paymentToken.transferFrom(
    msg.sender, address(this), want.mulDivDown(_toDVT(offer.price,
_currentRate), offer.totalShards)
);
```

So the amount paid per shard is $\text{price} * \text{rate} / (1\text{e}6 * \text{totalShards})$.

When the same buyer cancels that purchase, the refund is worked out with a completely different formula:

```
// src/shards/ShardsNFTMarketplace.sol:163
paymentToken.transfer(buyer,
purchase.shards.mulDivUp(purchase.rate, 1e6));
```

So the amount refunded per shard is $\text{rate} / 1\text{e}6$.

These two numbers are only equal when $\text{price} / \text{totalShards} == 1$, that is when $\text{price} == \text{totalShards}$. In this deployment the price is $1_000_000\text{e}6$ and the total shards is $10_000_000\text{e}18$, so the refund per shard is about $1\text{e}13$ times bigger than the amount that was paid per shard. The refund formula simply forgot to scale by the price and the total shards.

On top of that, the payment in `fill()` uses `mulDivDown`. For a small want the paid amount rounds all the way down to zero, so a buyer with no DVT at all can still open a purchase and then cancel it for a real refund. That refund is paid out of the DVT the marketplace holds (the seller fees), so the attacker walks away with the marketplace balance.

Location is `src/shards/ShardsNFTMarketplace.sol:135-137` for the payment and `src/shards/ShardsNFTMarketplace.sol:163` for the refund.

Impact:

Likelihood High

Risk High

Proof of Concept:

The player starts with zero DVT. They deploy one attacker contract. In its constructor it buys a few shards for zero DVT (the price rounds down), cancels to collect a small refund, then uses that seed money to buy a large chunk sized to the marketplace balance and cancels again to pull the whole balance out. Everything is sent to the recovery account. It is a single player transaction.

```
contract ShardsAttacker {
    constructor(ShardsNFTMarketplace marketplace,
        DamnValuableToken token, address recovery, uint64 offerId) {
        token.approve(address(marketplace), type(uint256).max);

        // 1. buy a few shards. The price paid rounds DOWN to zero
        uint256 idx0 = marketplace.fill(offerId, 100);

        // 2. cancel the buy. The refund uses a different formula,
        so we get real DVT back without paying anything
```



```
player nonce      : 1
```

Suite result: ok. 1 passed; 0 failed; 0 skipped

The marketplace starts with 750 DVT of fees and ends with only 74999999 wei. The recovery account receives 74999999999925000001 wei, which is basically the whole 750 DVT, and the staking balance is untouched.

Recommended Mitigation:

The refund must return exactly what the buyer paid, using the same formula. Store the paid amount on the purchase, or recompute the refund with the same price and total shards scaling.

The simplest fix is to save the DVT paid on the Purchase and refund that exact value.

File: src/shards/IShardsNFTMarketplace.sol, struct Purchase:

```
struct Purchase {
    uint256 shards;
    uint256 rate;
+   uint256 paid;
    uint64 timestamp;
    address buyer;
    bool cancelled;
}
```

File: src/shards/ShardsNFTMarketplace.sol, function fill():

```
+   uint256 paid = want.mulDivDown(_toDVT(offer.price,
    _currentRate), offer.totalShards);
    purchases[offerId].push(
        Purchase({
            shards: want,
            rate: _currentRate,
+           paid: paid,
            buyer: msg.sender,
            timestamp: uint64(block.timestamp),
            cancelled: false
        })
    );
-   paymentToken.transferFrom(
-       msg.sender, address(this),
    want.mulDivDown(_toDVT(offer.price, _currentRate),
    offer.totalShards)
-   );
+   paymentToken.transferFrom(msg.sender, address(this),
    paid);
```

File: src/shards/ShardsNFTMarketplace.sol, function cancel():

```
-      paymentToken.transfer(buyer,
+      purchase.shards.mulDivUp(purchase.rate, 1e6));
+      paymentToken.transfer(buyer, purchase.paid);
```

Medium

[M-1] fill() rounds the price down to zero, so shards can be bought for free

Description:

The DVT charged in fill() is rounded down:

```
// src/shards/ShardsNFTMarketplace.sol:135-137
paymentToken.transferFrom(
    msg.sender, address(this), want.mulDivDown(_toDVT(offer.price,
_currentRate), offer.totalShards)
);
```

Because totalShards is much larger than the price, small purchases make the calculation round down to 0 DVT. So, buying up to 133 shards costs nothing. An attacker can repeatedly buy small amounts for free and collect shards without paying. If an offer gets fully filled this way, the seller loses shards without receiving the expected DVT.

Impact:

Likelihood High, since it needs no permission and no funds. Risk Medium, because value only really moves once it is combined with the refund bug in [H-1] or when an offer closes.

Proof of Concept:

Step 1 of the [H-1] proof of concept is exactly this. The attacker calls fill(offerId, 100) while holding zero DVT and the call still succeeds, because the price rounds down to zero.

Recommended Mitigation:

Reject buys that would pay zero, and round the price up so the buyer never pays less than the value of the shards.

File: src/shards/ShardsNFTMarketplace.sol, function fill():

```
-      paymentToken.transferFrom(
-      msg.sender, address(this),
+      want.mulDivDown(_toDVT(offer.price, _currentRate),
+      offer.totalShards)
-      );
```



```
+      uint256 paid = want.mulDivUp(_toDVT(offer.price,
+      _currentRate), offer.totalShards);
+      if (paid == 0) revert BadAmount();
+      paymentToken.transferFrom(msg.sender, address(this),
+      paid);
```

[M-2] `cancel()` time window is inverted and does not match the documented behavior

Description:

The constants say a buyer must wait before they can cancel, and then has a period during which cancelling is allowed:

```
// src/shards/ShardsNFTMarketplace.sol:22-26
uint32 public constant TIME_BEFORE_CANCEL = 1 days;
uint32 public constant CANCEL_PERIOD_LENGTH = 2 days;
```

But the check inside `cancel()` reverts as soon as more than `TIME_BEFORE_CANCEL` has passed:

```
// src/shards/ShardsNFTMarketplace.sol:152-155
if (
    purchase.timestamp + CANCEL_PERIOD_LENGTH < block.timestamp
    || block.timestamp > purchase.timestamp +
    TIME_BEFORE_CANCEL
) revert BadTime();
```

The timing logic is reversed. You can only cancel within 1 day, instead of after waiting 1 day. Since you can cancel in the same block as the purchase, there is no real waiting period. This allows the attacker to complete the drain in one transaction, enabling [H-1].

Impact:

Likelihood High, Every cancel after the first day is affected. Risk Medium, since the cancel feature is broken for its intended use and the missing delay helps the theft in [H-1].

Proof of Concept:

N/A

Recommended Mitigation:

Allow cancelling only inside the intended window, from `timestamp + TIME_BEFORE_CANCEL` up to `timestamp + TIME_BEFORE_CANCEL + CANCEL_PERIOD_LENGTH`.

File: `src/shards/ShardsNFTMarketplace.sol`, function `cancel()`:

```

-         if (
-             purchase.timestamp + CANCEL_PERIOD_LENGTH <
-             block.timestamp
-             || block.timestamp > purchase.timestamp +
-             TIME_BEFORE_CANCEL
-         ) revert BadTime();
+         uint256 start = purchase.timestamp + TIME_BEFORE_CANCEL;
+         if (block.timestamp < start || block.timestamp > start +
+             CANCEL_PERIOD_LENGTH) revert BadTime();

```

Low

[L-1] Missing zero address check on the oracle in the constructor

Description:

The constructor stores the oracle address without checking it, and the oracle is the only account that can update the rate.

```

// src/shards/ShardsNFTMarketplace.sol:44-49
address _oracle,
uint256 _initialRate
) ERC1155("") {
    ...
    oracle = _oracle;

```

Impact:

Likelihood Low Risk Low.

Proof of Concept:

N/A

Recommended Mitigation:

Add a zero address check in the constructor.

```

+         if (_oracle == address(0)) revert NotAllowed();
+         oracle = _oracle;

```

Informational

[I-1] Unused custom errors in the interface

Description:

The interface declares several errors that are never used anywhere, such as `TooManyShards`, `MustPayFee`, and `UnknownOffer` variants. Dead declarations add noise and can confuse readers about what the code actually checks. Aderyn flags these.

```
// src/shards/IShardsNFTMarketplace.sol:34-38
error TooManyShards();
error MustPayFee();
```

Location is `src/shards/IShardsNFTMarketplace.sol:34-38`.

Proof of Concept:

N/A

Recommended Mitigation:

Remove the errors that are never used.

[I-2] Magic numbers used instead of named constants

Description:

The math uses raw literals like `1e6`, `100e6`, and `1e6` for the fee scaling and the rate scaling in more than one place. Aderyn flags these. Named constants make the intent clear and stop the values from drifting apart.

```
// src/shards/ShardsNFTMarketplace.sol:163
mulDivUp(purchase.rate, 1e6)
// src/shards/ShardsNFTMarketplace.sol:180  mulDivDown(1e6, 100e6)
// src/shards/ShardsNFTMarketplace.sol:219  mulDivDown(_rate, 1e6)
```

Proof of Concept:

N/A

Recommended Mitigation:

Declare named constants, eg. `uint256 constant RATE_SCALE = 1e6;` and `uint256 constant FEE_DENOMINATOR = 100e6;`, and use it in formulas.